

SAMPLE QUERIES
FOR
Deen Associate Management System

VERSION 1.0

Prepared By
Muhammad Junaid Riaz

27th November, 2025

Revision History

Version	Description	Author	Date
1.0	This document contains sample database queries for Deen Associate Management System (DAMS).	Muhammad Junaid Riaz	27th November, 2025

Sample Queries

This section presents sample queries used by the Deen Associate Management System (DAMS). The application implements these operations using Entity Framework Core (LINQ) in backend services. The SQL statements below show equivalent database operations against Microsoft SQL Server.

Query Summary Table

Query No.	Query Name	Service Method	API Endpoint
1	User Login Verification	AuthService.Login()	POST /api/Auth/login
2	Check Duplicate Email on Registration	AuthService.RegisterAsync()	POST /api/Auth/register
3	Insert New Client User	AuthService.RegisterAsync()	POST /api/Auth/register
4	Retrieve All Projects	ProjectService.GetAllProjectsAsync()	GET /api/Project
5	Retrieve Project by ID	ProjectService.GetProjectByIdAsync()	GET /api/Project/{id}
6	Retrieve Units by Project	UnitService.GetUnitsByProjectIdAsync()	GET /api/Unit/project/{projectId}
7	Search and Filter Customers	CustomerService.GetCustomersAsync()	GET /api/Customer?search=&source=&status=
8	Find Existing Customer (Deduplication)	CustomerService.FindOrCreateCustomerAsync()	Used internally by BookingService / BookingRequestService
9	Insert Booking Request	BookingRequestService.SubmitBookingRequestAsync()	POST /api/BookingRequest
10	List Booking Requests (Admin)	BookingRequestService.GetBookingRequestsAsync()	GET /api/BookingRequest?status=&projectId=&search=
11	Approve Booking Request	BookingRequestService.ApproveBookingRequestAsync()	POST /api/BookingRequest/{id}/approve
12	Create Walk-in Booking	BookingService.CreateBookingAsync()	POST /api/Booking
13	List Bookings with Search and Pagination	BookingService.GetBookingsAsync()	GET /api/Booking?status=&projectId=&search=
14	Client View Own Bookings (My Projects)	BookingService.GetBookingsByCustomerEmailAsync()	GET /api/MyProjects
15	Record Booking Amount Payment	BookingService.RecordBookingPaymentAsync()	POST /api/Booking/{id}/payments

Query No.	Query Name	Service Method	API Endpoint
16	Retrieve Installment Schedule	InstallmentService.GetScheduleAsync()	GET /api/Booking/{id}/installments
17	Record Installment Payment	InstallmentService.RecordInstallmentPaymentAsync()	POST /api/Booking/{id}/installments/{installmentId}/payment
18	Finance Dashboard — Automatic Revenue	FinanceService.GetDashboardAsync()	GET /api/Finance/dashboard?projectId=&from=&to=
19	Finance Dashboard — Manual Revenue	FinanceService.GetDashboardAsync()	GET /api/Finance/dashboard?projectId=&from=&to=
20	Finance Dashboard — Expenses	FinanceService.GetDashboardAsync()	GET /api/Finance/dashboard?projectId=&from=&to=
21	List Employees with Filters	EmployeeService.GetAllEmployeesAsync()	GET /api/Employee?department=&status=
22	Record Employee Attendance	EmployeeService.RecordAttendanceAsync()	POST /api/Employee/{id}/attendance
23	Generate Employee Salary	EmployeeService.GenerateSalaryAsync()	POST /api/Employee/{employeeId}/salary
24	Cancel Booking and Release Unit	BookingService.CancelBookingAsync()	POST /api/Booking/{id}/cancel
25	Booking Request Statistics	BookingRequestService.GetBookingRequestStatsAsync()	GET /api/BookingRequest/stats

Detailed Sample Queries

Query 1: User Login Verification

Description: Verify registered user credentials during login.

Service / Endpoint: AuthService.Login() | POST /api/Auth/login

Equivalent SQL Query:

```
SELECT u.UserId, u.FullName, u.Email, u.Password, r.Role_name
FROM Users u
INNER JOIN Roles r ON u.RoleId = r.RoleId
WHERE u.Email = @Email;
```

Query 2: Check Duplicate Email on Registration

Description: Prevent duplicate client registration using the same email address.

Service / Endpoint: AuthService.RegisterAsync() | POST /api/Auth/register

Equivalent SQL Query:

```
SELECT UserId
FROM Users
WHERE LOWER(Email) = LOWER(@Email);
```

Query 3: Insert New Client User

Description: Register a new client user account with hashed password.

Service / Endpoint: AuthService.RegisterAsync() | POST /api/Auth/register

Equivalent SQL Query:

```
INSERT INTO Users (FullName, Email, Password, RoleId)
VALUES (@FullName, @Email, @HashedPassword, @ClientRoleId);
```

Query 4: Retrieve All Projects

Description: Fetch all real-estate projects for public listing and admin dashboard.

Service / Endpoint: ProjectService.GetAllProjectsAsync() | GET /api/Project

Equivalent SQL Query:

```
SELECT Id, ProjectName, Location, Description, StartingDate,
ExpectedCompletionDate, Status, CreatedAt
FROM Projects
ORDER BY CreatedAt DESC;
```

Query 5: Retrieve Project by ID

Description: Fetch details of a single project.

Service / Endpoint: ProjectService.GetProjectByIdAsync() | GET /api/Project/{id}

Equivalent SQL Query:

```
SELECT Id, ProjectName, Location, Description, StartingDate,
       ExpectedCompletionDate, Status, CreatedAt
FROM Projects
WHERE Id = @ProjectId;
```

Query 6: Retrieve Units by Project

Description: List all units belonging to a selected project.

Service / Endpoint: UnitService.GetUnitsByProjectIdAsync() | GET /api/Unit/project/{projectId}

Equivalent SQL Query:

```
SELECT Id, ProjectId, UnitNumber, UnitType, FloorNumber, Size, Price, Status
FROM Units
WHERE ProjectId = @ProjectId
ORDER BY FloorNumber, UnitNumber;
```

Query 7: Search and Filter Customers

Description: Admin searches customers by name, phone, CNIC, or email with optional filters.

Service / Endpoint: CustomerService.GetCustomersAsync() | GET /api/Customer?search=&source=&status=

Equivalent SQL Query:

```
SELECT c.*, COUNT(b.Id) AS BookingsCount
FROM Customers c
LEFT JOIN Bookings b ON b.CustomerId = c.Id
WHERE (@Source IS NULL OR c.Source = @Source)
AND (@Status IS NULL OR c.Status = @Status)
AND (
  LOWER(c.FullName) LIKE '%' + LOWER(@SearchTerm) + '%'
OR c.Phone LIKE '%' + @SearchTerm + '%'
OR c.CNIC LIKE '%' + @SearchTerm + '%'
OR LOWER(c.Email) LIKE '%' + LOWER(@SearchTerm) + '%'
)
GROUP BY c.Id
ORDER BY c.CreatedAt DESC
OFFSET @Skip ROWS FETCH NEXT @PageSize ROWS ONLY;
```

Query 8: Find Existing Customer (Deduplication)

Description: Locate an existing customer by CNIC, then phone, then email before creating a new record.

Service / Endpoint: CustomerService.FindOrCreateCustomerAsync() | Used internally by BookingService / BookingRequestService

Equivalent SQL Query:

```
SELECT TOP 1 Id FROM Customers WHERE CNIC = @CNIC;
-- if not found:
SELECT TOP 1 Id FROM Customers WHERE Phone = @Phone;
-- if not found:
SELECT TOP 1 Id FROM Customers WHERE Email = @Email;
```

Query 9: Insert Booking Request

Description: Client submits an online booking request for an available unit.

Service / Endpoint: BookingRequestService.SubmitBookingRequestAsync() | POST /api/BookingRequest

Equivalent SQL Query:

```
INSERT INTO BookingRequests
  (UnitId, FullName, Phone, CNIC, Email, Address, Status, UserId, CreatedAt)
VALUES
  (@UnitId, @FullName, @Phone, @CNIC, @Email, @Address, 'Pending', @UserId, GETUTCDATE());
```

Query 10: List Booking Requests (Admin)

Description: Admin views booking requests with project, unit, and status filters.

Service / Endpoint: BookingRequestService.GetBookingRequestsAsync() | GET /api/BookingRequest?status=&projectId=&search=

Equivalent SQL Query:

```
SELECT br.*, u.UnitNumber, p.ProjectName
FROM BookingRequests br
INNER JOIN Units u ON br.UnitId = u.Id
INNER JOIN Projects p ON u.ProjectId = p.Id
WHERE (@Status IS NULL OR br.Status = @Status)
  AND (@ProjectId IS NULL OR u.ProjectId = @ProjectId)
ORDER BY br.CreatedAt DESC
OFFSET @Skip ROWS FETCH NEXT @PageSize ROWS ONLY;
```

Query 11: Approve Booking Request

Description: Admin approves a pending request, creates customer/booking, and marks unit as booked.

Service / Endpoint: BookingRequestService.ApproveBookingRequestAsync() | POST /api/BookingRequest/{id}/approve

Equivalent SQL Query:

```
UPDATE BookingRequests
SET Status = 'Approved', ReviewedByUserId = @AdminUserId, ReviewedAt = GETUTCDATE()
WHERE Id = @BookingRequestId AND Status = 'Pending';

INSERT INTO Bookings (CustomerId, UnitId, BookingRequestId, Status, ...)
VALUES (@CustomerId, @UnitId, @BookingRequestId, 'AwaitingBookingAmount', ...);

UPDATE Units SET Status = 'Booked' WHERE Id = @UnitId;
```

Query 12: Create Walk-in Booking

Description: Admin creates a booking directly for an available unit.

Service / Endpoint: BookingService.CreateBookingAsync() | POST /api/Booking

Equivalent SQL Query:

```

SELECT u.*, p.ProjectName
FROM Units u
INNER JOIN Projects p ON u.ProjectId = p.Id
WHERE u.Id = @UnitId AND u.Status = 'Available';

INSERT INTO Bookings
  (CustomerId, UnitId, Status, AgreedSalePrice, BookingAmountRequired, CreatedByUserId, CreatedAt)
VALUES
  (@CustomerId, @UnitId, 'AwaitingBookingAmount', @AgreedSalePrice, @BookingAmountRequired,
  @AdminUserId, GETUTCDATE());

UPDATE Units SET Status = 'Booked' WHERE Id = @UnitId;

```

Query 13: List Bookings with Search and Pagination

Description: Admin retrieves bookings filtered by status, project, customer, or search term.

Service / Endpoint: BookingService.GetBookingsAsync() | GET /api/Booking?status=&projectId=&search=

Equivalent SQL Query:

```

SELECT b.*, c.FullName, u.UnitNumber, p.ProjectName
FROM Bookings b
INNER JOIN Customers c ON b.CustomerId = c.Id
INNER JOIN Units u ON b.UnitId = u.Id
INNER JOIN Projects p ON u.ProjectId = p.Id
WHERE (@Status IS NULL OR b.Status = @Status)
  AND (@ProjectId IS NULL OR u.ProjectId = @ProjectId)
  AND (
    LOWER(b.BookingReference) LIKE '%' + LOWER(@SearchTerm) + '%'
    OR LOWER(c.FullName) LIKE '%' + LOWER(@SearchTerm) + '%'
    OR LOWER(u.UnitNumber) LIKE '%' + LOWER(@SearchTerm) + '%'
  )
ORDER BY b.BookingDate DESC
OFFSET @Skip ROWS FETCH NEXT @PageSize ROWS ONLY;

```

Query 14: Client View Own Bookings (My Projects)

Description: Logged-in client views bookings linked to the same email as the JWT account.

Service / Endpoint: BookingService.GetBookingsByCustomerEmailAsync() | GET /api/MyProjects

Equivalent SQL Query:

```

SELECT b.*, c.FullName, u.UnitNumber, p.ProjectName
FROM Bookings b
INNER JOIN Customers c ON b.CustomerId = c.Id
INNER JOIN Units u ON b.UnitId = u.Id
INNER JOIN Projects p ON u.ProjectId = p.Id
WHERE LOWER(c.Email) = LOWER(@JwtEmail)
  AND b.Status <> 'Cancelled'
ORDER BY b.BookingDate DESC;

```

Query 15: Record Booking Amount Payment

Description: Admin records initial booking amount payment and updates booking totals.

Service / Endpoint: BookingService.RecordBookingPaymentAsync() | POST /api/Booking/{id}/payments

Equivalent SQL Query:


```

INSERT INTO Payments
  (BookingId, Type, Amount, PaymentMethod, Reference, PaidAt, ReceiptNumber, CreatedByUserId)
VALUES
  (@BookingId, 'BookingAmount', @Amount, @PaymentMethod, @Reference, @PaidAt, @ReceiptNumber,
  @AdminUserId);

UPDATE Bookings
SET BookingAmountReceived = BookingAmountReceived + @Amount,
    Status = CASE WHEN BookingAmountReceived + @Amount >= BookingAmountRequired
    THEN 'PaymentPlanActive' ELSE Status END
WHERE Id = @BookingId;

```

Query 16: Retrieve Installment Schedule

Description: Fetch installment plan rows for a booking.

Service / Endpoint: InstallmentService.GetScheduleAsync() | GET /api/Booking/{id}/installments

Equivalent SQL Query:

```

SELECT i.*
FROM Installments i
INNER JOIN Bookings b ON i.BookingId = b.Id
WHERE b.Id = @BookingId
ORDER BY i.SequenceNumber;

```

Query 17: Record Installment Payment

Description: Admin records payment against a specific installment.

Service / Endpoint: InstallmentService.RecordInstallmentPaymentAsync() | POST /api/Booking/{id}/installments/{installmentId}/payment

Equivalent SQL Query:

```

INSERT INTO Payments
  (BookingId, InstallmentId, Type, Amount, PaymentMethod, Reference, PaidAt, ReceiptNumber)
VALUES
  (@BookingId, @InstallmentId, 'Installment', @Amount, @PaymentMethod, @Reference, @PaidAt,
  @ReceiptNumber);

UPDATE Installments
SET Status = CASE WHEN @RemainingAfterPayment <= 0 THEN 'Paid' ELSE 'PartiallyPaid' END
WHERE Id = @InstallmentId;

```

Query 18: Finance Dashboard — Automatic Revenue

Description: Aggregate booking and installment payments as automatic revenue.

Service / Endpoint: FinanceService.GetDashboardAsync() | GET /api/Finance/dashboard?projectId=&from=&to=

Equivalent SQL Query:

```

SELECT p.PaidAt, p.Amount, p.Type, p.ReceiptNumber,
       b.BookingReference, c.FullName, pr.ProjectName
FROM Payments p
INNER JOIN Bookings b ON p.BookingId = b.Id
INNER JOIN Customers c ON b.CustomerId = c.Id
INNER JOIN Units u ON b.UnitId = u.Id
LEFT JOIN Projects pr ON u.ProjectId = pr.Id
WHERE b.Status <> 'Cancelled'
      AND (@ProjectId IS NULL OR u.ProjectId = @ProjectId)
      AND (@FromDate IS NULL OR p.PaidAt >= @FromDate)
      AND (@ToDate IS NULL OR p.PaidAt < @ToDateExclusive);

```

Query 19: Finance Dashboard — Manual Revenue

Description: Retrieve manually entered project revenue records.

Service / **Endpoint:** FinanceService.GetDashboardAsync() | GET
 /api/Finance/dashboard?projectId=&from=&to=

Equivalent SQL Query:

```

SELECT mr.Id, mr.Date, mr.RevenueType, mr.Amount, mr.Reference, mr.Description,
       p.ProjectName
FROM ManualRevenues mr
LEFT JOIN Projects p ON mr.ProjectId = p.Id
WHERE (@ProjectId IS NULL OR mr.ProjectId = @ProjectId)
      AND (@FromDate IS NULL OR mr.Date >= @FromDate)
      AND (@ToDate IS NULL OR mr.Date < @ToDateExclusive);

```

Query 20: Finance Dashboard — Expenses

Description: Retrieve project and general expenses including salary expenses.

Service / **Endpoint:** FinanceService.GetDashboardAsync() | GET
 /api/Finance/dashboard?projectId=&from=&to=

Equivalent SQL Query:

```

SELECT e.Id, e.Date, e.Category, e.Amount, e.Vendor, e.Description,
       p.ProjectName
FROM Expenses e
LEFT JOIN Projects p ON e.ProjectId = p.Id
WHERE (@ProjectId IS NULL OR e.ProjectId = @ProjectId)
      AND (@FromDate IS NULL OR e.Date >= @FromDate)
      AND (@ToDate IS NULL OR e.Date < @ToDateExclusive);

```

Query 21: List Employees with Filters

Description: Admin retrieves employees filtered by department and status.

Service / Endpoint: EmployeeService.GetAllEmployeesAsync() | GET /api/Employee?department=&status=

Equivalent SQL Query:

```
SELECT Id, FullName, Department, Salary, Status, JoinDate
FROM Employees
WHERE (@Department IS NULL OR Department = @Department)
AND (@Status IS NULL OR Status = @Status)
ORDER BY FullName;
```

Query 22: Record Employee Attendance

Description: Mark daily attendance for an employee on a selected date.

Service / Endpoint: EmployeeService.RecordAttendanceAsync() | POST /api/Employee/{id}/attendance

Equivalent SQL Query:

```
SELECT Id FROM EmployeeAttendances
WHERE EmployeeId = @EmployeeId AND [Date] = @Date;

-- if not exists:
INSERT INTO EmployeeAttendances (EmployeeId, Date, Status, Notes, RecordedByUserId)
VALUES (@EmployeeId, @Date, @Status, @Notes, @AdminUserId);
```

Query 23: Generate Employee Salary

Description: Create monthly salary record and linked expense entry.

Service / Endpoint: EmployeeService.GenerateSalaryAsync() | POST /api/Employee/{employeeId}/salary

Equivalent SQL Query:

```
SELECT Id FROM EmployeeSalaries
WHERE EmployeeId = @EmployeeId AND PayMonth = @Month AND PayYear = @Year;

INSERT INTO Expenses (Category, Amount, Vendor, Date, ProjectId, CreatedByUserId)
VALUES ('Salary', @Amount, @EmployeeName, @PayDate, @ProjectId, @AdminUserId);

INSERT INTO EmployeeSalaries (EmployeeId, Amount, PayMonth, PayYear, ExpenseId, PayDate)
VALUES (@EmployeeId, @Amount, @Month, @Year, @ExpenseId, @PayDate);
```

Query 24: Cancel Booking and Release Unit

Description: Admin cancels an active booking and returns the unit to Available status.

Service / Endpoint: BookingService.CancelBookingAsync() | POST /api/Booking/{id}/cancel

Equivalent SQL Query:

```
UPDATE Bookings
SET Status = 'Cancelled', UpdatedAt = GETUTCDATE(), InternalNotes = @UpdatedNotes
WHERE Id = @BookingId AND Status NOT IN ('Cancelled', 'PossessionGiven', 'SaleCompleted');

UPDATE Units
SET Status = 'Available', UpdatedAt = GETUTCDATE()
WHERE Id = @UnitId;
```

Query 25: Booking Request Statistics

Description: Count booking requests grouped by status for admin dashboard cards.

Service / **Endpoint:** BookingRequestService.GetBookingRequestStatsAsync() | GET
/api/BookingRequest/stats

Equivalent SQL Query:

```
SELECT Status, COUNT(*) AS RequestCount  
FROM BookingRequests  
GROUP BY Status;
```